

Spark vs. Ray

Name: Sidhartha Durgam
Roll Number: DA24B003

Measured comparison of NYC Yellow Taxi preprocessing on an Ubuntu ARM64 VM. Evidence: assignment PDF, student-pasted terminal outputs, recovered Ray logs, and original UI screenshots [A, E].

Finding. Full data: Spark UDF **612.13 s**, native **570.36 s**; Ray **2262.83 s** reconstructed successful processing. Both produced **121,692,576 rows**; the descriptive **3.70x** ratio has different timing scopes. A new **200,000-input-row subset** matched all 17 output columns and duplicate counts across three variants (**197,338 rows** each); Ray was fastest on that subset (**10.25 s**) [Q]. Neither result is a universal ranking.

1. Dataset and cluster configuration

Input: **35 monthly Parquet files**, February 2023 through December 2025, **125,135,782 rows**, and **1.97 GiB** of file bytes. The taxi-zone CSV contains **265 unique LocationIDs**. Data comes from NYC TLC [1]. Both clusters ran separately on the same VM: Ubuntu 26.04 LTS, aarch64, 4 virtual CPUs, about 3.3 GiB RAM, 3.8 GiB swap, and a 39G root filesystem [E]. These are colocated worker nodes/processes. The supplied final launcher sets head logical memory to 512 MiB; earlier setup logs used 256 MiB, so session configurations varied.

Spark 3.5.7 / Python 3.11.17	Ray 2.59.0 / Python 3.11.17
1 master + 2 workers, each 1 core / 768 MiB advertised. Each executor: 1 core / 512 MiB; driver: 512 MiB; total executor cores: 2; shuffle partitions: 32; Java 17.0.20.1.	1 head (0 task CPUs) + 2 worker nodes (1 CPU each), launched using Cluster(). Supplied final launcher: head 512 MiB logical memory / 80 MiB object store; each worker 650 / 120 MiB. Total object store: 320 MiB.

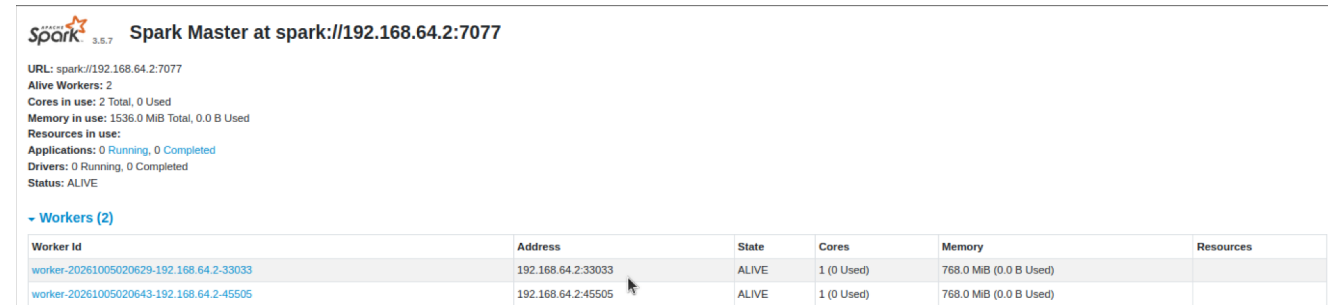


Figure 1. Spark Master on port 8080: two ALIVE workers, each 1 core and 768 MiB advertised. Screenshot before native runs; executor allocations are verified by the terminal logs.

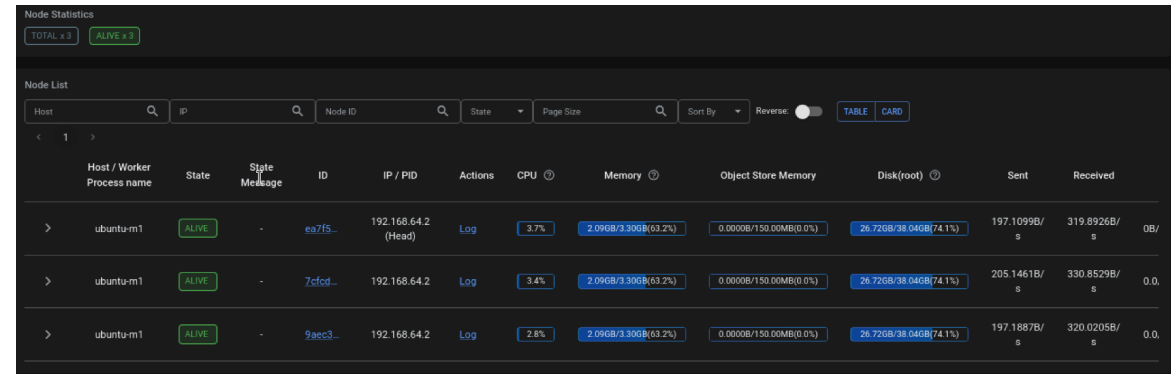


Figure 2. Ray Dashboard on port 8265: three ALIVE nodes, one Head and two workers. This earlier setup screenshot shows 150 MB object stores per node; it is not the final 80/120/120 MiB configuration.

IMPLEMENTATION & VALIDATION

What each pipeline performed

Both scripts read the same trip files and lookup CSV, select nine common source columns, and export 17 named fields. The 2025-only congestion-fee column was omitted consistently. The intended flow is ingestion → cleansing → timestamps → two location joins → speed feature → Parquet export [E].

Step	Spark implementation	Ray implementation used
Ingestion / nulls	Read shared Parquet directory; drop nulls in five required fields.	Read each monthly file via ray.data; pandas notna() mask on the same five fields.
Duplicates / validity	Global dropDuplicates() over the nine selected fields; distance > 0 and dropoff > pickup.	pandas drop_duplicates() within each month/block; same distance and timestamp filters.
Timestamps / duration	Format yyyy-MM-dd HH:mm:ss; cast timestamps to timestamp then integer seconds; divide elapsed seconds by 3600.	pandas strftime to the same text format; elapsed total_seconds()/3600.
Trip / location joins	Two left joins: pickup ID and dropoff ID to renamed lookup fields.	Small lookup captured in batch functions; two local pandas left merges (broadcast-style).
Python feature / export	Scalar F.udf returning double: miles / duration_hours; write Parquet. Native experiment uses column division.	map_batches Python function with NumPy divide, guarding duration > 0; write monthly Parquet.

Required null checks cover pickup/dropoff timestamps, distance and both location IDs. Optional nulls are retained; lookup parsing preserves literal "N/A". Both joins use large trips and a tiny dimension, not a demonstrated large-to-large shuffle-join stress test.

2. Output validation: confirmed facts

Validation	Spark Python UDF	Ray
Rows / Parquet files	121,692,576 / 32	121,692,576 / 35
Output footprint	3.8G (du disk allocation)	3.516 GiB (sum of file bytes)
Schema inspected	17 fields; IDs int32, passenger_count int64, numeric features double, text string.	Same names and numeric types in inspected schema; text large_string.

Combined full-data removal: **3,443,206 rows (2.75%)**; per-filter counts unavailable. The footprints use different measurement methods. String/large_string is an Arrow storage representation difference.

New subset validation [Q]. The same first **200,000 rows of one monthly file** were processed using the submitted transformations. All variants exported **197,338 rows**. SQLite full-row multiset checks over **all 17 columns** passed for Spark UDF/native and Spark UDF/Ray: **zero unmatched instances both ways**. Duplicate counts were preserved; file/row order and integer-width/text-storage differences ignored. Floats used exact hex values without tolerance; signed zero and NaN/null were distinguished.

Full-data limitation: the nonrandom subset cannot expose cross-month duplicates. Global versus monthly/block deduplication and integer-second versus fractional durations remain potential differences. Full-data record equality and native parity are unverified.

The structured export still needs model-specific encoding, optional-null handling and leakage-safe train/test splits. No model training was benchmarked.

BENCHMARK RESULTS

Runtime and Python UDF analysis

Original full-data result	Seconds	Measurement scope
Spark + Python UDF	612.13	One successful ingestion-to-export run (saved: 612.133718 s).
Spark native expression	570.36	Successful rerun with the same settings.
Ray successful processing	2262.83	35-month reconstruction; mixed timing scopes.
Ray last resumed session	1554.14	Final session only (saved: 1554.136178 s).

Full-data reported processing time (seconds; lower is faster)

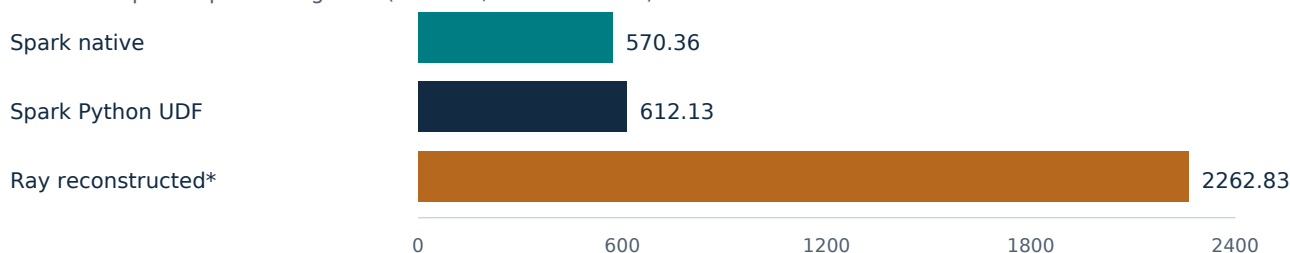


Figure 3. *Ray combines successful dataset execution intervals and per-month wall-clock intervals. Bars are a descriptive comparison, not identical timer boundaries or a controlled repeated-run study.

3. Ray runtime reconstruction and resumptions

Feb-Nov 2023: ten recovered successful dataset execution intervals, with write included, sum to **709.74 s**.
Dec 2023-Dec 2025: 25 unique Completed-month wall-clock entries sum to **1553.09 s**. Total: **2262.83 s (37.71 min)**. Duplicate log copies and the earlier smoke test were excluded. This mixed-timer sum omits failed/retry work, restart downtime, gaps and some setup overhead. Completed month folders were reused; incomplete folders were removed manually before resuming. Full operational elapsed time remains unknown. The 35-month reconstruction CSV preserves each interval [E].

4. Observed UDF-associated end-to-end difference

612.13 - 570.36 = 41.77 s; $41.77 / 570.36 \times 100 = 7.32\%$; runtime ratio $612.13 / 570.36 = 1.073x$.
This is the observed whole-pipeline penalty associated with the scalar Python UDF variant, not a direct measurement of 41.77 s spent inside the UDF or solely at the JVM boundary. One successful run per variant cannot separate cache, I/O, swap and scheduling noise.

Supplementary subset [Q]	Pipeline (s)	Feature median (s)	Output rows
Spark Python UDF	21.892369	3.045933	197,338
Spark native	15.953042	0.780352	197,338
Ray NumPy batch	10.245127	0.385089	197,338

Subset pipeline timers span ingestion-to-export after engine initialization, including lookup loading. Separate feature trials use cached joined input: one warmup, then three timed materializations of distance, duration and speed. Spark uses persist/count; Ray uses materialize; an untimed cached sum consumes speed values. These measure framework execution, scheduling, conversion and cache population, **not pure function-body time**. One pipeline run per variant and the differing sinks limit causal comparison [Q].

Ray's vectorized NumPy feature stage was faster in this subset than Spark's scalar Python UDF. Spark native execution was also faster than its UDF. This supports a workload-specific Python/batch execution advantage, not a universal framework result; Spark supports Arrow/vectorized UDF alternatives [2, 3].

RESOURCE EVIDENCE

Samples and monitored subset maxima

Original full-data runs lack continuous monitoring and verified run-wide peaks. The historical Spark top sample is from the native attempt that later failed for disk capacity. The supplementary runs add nominal **0.5-second whole-VM monitoring** during each subset ingestion-to-export interval [E, Q].

```
top - 02:16:24 up 7:34, 1 user, load average: 2.61, 2.26, 1.80
Tasks: 258 total, 1 running, 250 sleeping, 0 stopped, 7 zombie
%Cpu(s): 51.4 us, 9.0 sy, 0.0 ni, 39.2 id, 0.4 wa, 0.0 hi, 0.0 si, 0.0 st
MiB Mem : 3380.7 total, 205.8 free, 2641.8 used, 708.0 buff/cache
MiB Swap: 3892.0 total, 1618.4 free, 2273.6 used, 738.9 avail Mem
```

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
110650	sidhart+	20	0	4345780	726572	15356	S	99.7	21.0	6:27.31	java
110647	sidhart+	20	0	4344952	643552	15420	S	91.8	18.6	6:14.00	java
40517	sidhart+	20	0	3585792	180952	48724	S	7.2	5.2	26:49.04	firefox

Figure 4. Cropped original top sample over Spark UI. Memory values are in MiB; process CPU percentages and whole-machine CPU percentages have different denominators.

Historical snapshot [E]	Observed values	Interpretation
Spark failed native attempt	CPU: 51.4% user + 9.0% system. RAM: 2641.8/3380.7 MiB. Swap: 2273.6/3892.0 MiB used.	60.4% user+system sample. Swap 1618.4 MiB was FREE, correcting the earlier chat. These are not successful full-run peaks.
Ray active-run overview	1/2 logical CPU; 65.41 KiB / 320 MiB object store.	Scheduling/object occupancy. Dashboard 0B logical memory does not mean zero physical RAM.
Ray earlier cluster view	About 2.09 / 3.30 GB host memory.	Three colocated node rows repeat the same host; do not sum them. Earlier configuration.

Subset pipeline [Q]	Host CPU max observed (%)	Host RAM max observed (MiB)
Spark UDF	99.505	2917.418
Spark native	99.000	2791.570
Ray	98.515	2664.785

These are **sampled maxima for the subset**, including OS/browser/monitor and other processes, not framework-only or instantaneous peaks. CPU uses aggregate /proc/stat busy time on a 0-100% scale across all virtual CPUs, excluding iowait. RAM is MemTotal minus MemAvailable, unlike the historical top accounting. Ray's lower observed RAM maximum applies only to these runs; baselines and single-run noise constrain efficiency claims [Q].

5. Failures, capacity and effective parallelism

Spark initially used a 1.7G RAM-backed /tmp area; spill was moved onto the disk-backed root filesystem. A native attempt later failed with **No space left on device**. After recovery (9.3G available), the unchanged benchmark completed in 570.36 s. Failed runs are excluded from the timing table.

Ray global deduplication/shuffle-join trials stalled. The completed path used monthly blocks and local merges, with resumptions after GCS/Raylet failures. A spill warning reported a filesystem over 95% full: storage pressure, not a RAM peak. The logs do not prove an OOM-killer cause for every termination.

Two Ray worker CPUs were available, but the monthly path showed one active task and only one write input; the subset run repeats that warning. Near-100% whole-VM CPU does not prove both Ray workers were busy. Historical dashboard charts were absent without Prometheus/Grafana; supplementary /proc traces provide separate monitoring [E, Q].

DISCUSSION & ATTRIBUTION

Winner, project choice and fair use

6. Which framework won this data-engineering duel?

Full-data practical winner: Spark. Its UDF run took 10.20 minutes; Ray's reconstructed successful sum was 37.71 minutes, a descriptive **3.697x** ratio. Spark completed the whole dataset in one successful run; Ray needed recovery. **Subset winner: Ray.** On the same saved 200,000 input rows, Ray took **10.25 s**, Spark native **15.95 s** and Spark UDF **21.89 s**, with matching outputs. This reversal shows sensitivity to scale, partitioning, startup/caching and memory pressure. One run per variant and differing full-data timers prevent a universal ranking [E, Q].

A plausible explanation is Spark's optimized structured execution and stronger use of the two executor cores for this ETL task, while Ray's one-block monthly path restricts concurrent work and repeatedly uses pandas/Arrow conversions and DataFrame copies. Spark supports broadcast joins and adaptive query execution [4], but the final physical join plan was not preserved; no particular Spark join strategy is asserted.

7. AI-first versus BI-first projects

BI-first: choose Spark for this measured workload of tabular cleansing, deduplication, joins and batch export. Native expressions also reduced observed runtime. **AI-first:** consider Ray for Python model preprocessing/inference, stateful model workers and flexible tasks. Ray documents batch functions and reusable inference actors [3, 5]. No GPU inference or distributed training was tested here, so this is a project-design recommendation, not an AI-workload benchmark result. A combined design could use Spark for structured ETL and Ray for model workloads, subject to measuring transfer cost.

8. Performance Tuning Note: LLM assistance

Attribution: ChatGPT supplied pipeline code/translation and suggested **broadcast-style pandas lookup merges** to reduce shuffle pressure, **one month/block at a time**, smaller memory/object stores, disk-backed spill paths and resumptions. These choices supported completion but reduced Ray parallelism and weakened global parity; no isolated before/after Ray tuning speedup was measured. It suggested replacing Spark's speed UDF with native division: full-data runtime fell from 612.13 to 570.36 s, an observed whole-pipeline difference. ChatGPT also supplied the supplementary profiling/comparison helper; the student generated its results by running it in the VM. Feature-stage medians and their overhead limits are reported separately on p. 3 [E, Q].

9. Assignment coverage and remaining evidence gaps

Pipeline and orchestration: pp. 1-2. Subset exact-value/native parity: p. 2. Full-data reconstruction and separately executed subset feature stages: p. 3. Historical samples and monitored subset maxima: p. 4. Winner, AI-first/BI-first choice and attribution: p. 5. **Full-dataset record parity, full-run resource peaks, original full-data isolated feature times and full Ray operational elapsed time remain unverified.** The subset evidence strengthens these topics without retrospectively establishing the missing full-data measurements.

Sources. [A] DA3408 A3, Assignment 3-1.pdf, pp. 1-3. [E] Ubuntu VM Setup and Comparison: original terminal outputs and seven screenshots, 4-5 Oct 2026; supplied scripts and recovered Ray logs. [Q] Supplied quick-check evidence archive, run 20261004T232008998532Z: original metrics, logs, /proc resource traces, generated instrumented scripts and comparison summary. Recorded medians/maxima were checked against the raw traces. Parquet outputs were not included, so the reported multiset result could not be recomputed independently; deleted full-data outputs were not revalidated.

[1] NYC TLC Trip Record Data. [2] Spark 3.5.7: Arrow and Pandas UDFs. [3] Ray Data: map_batches. [4] Spark 3.5.7: performance tuning. [5] Ray overview. Official documentation checked 5 Oct 2026.